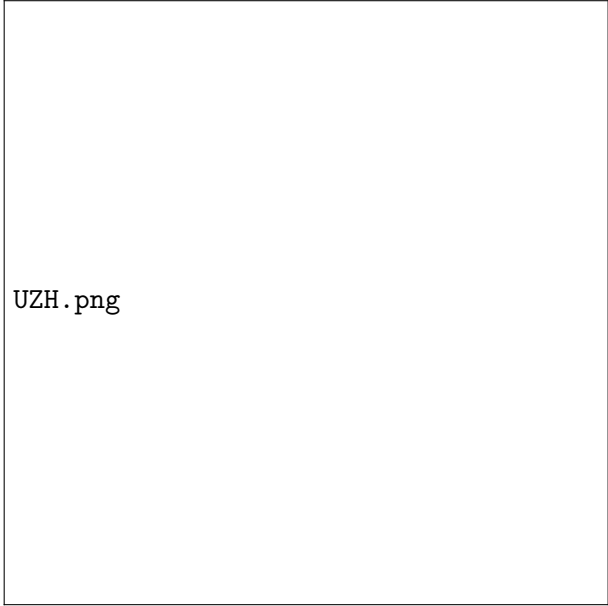




UZH.png

Efficient neural network wave functions for interacting fermions

Bachelor Thesis in Physics

Illia Yakushevskyi

Supervised by
Prof. Dr. Titus Neupert
Dr. Attila Szabó
Ph.D. candidate Simon Hille

July 22, 2026

Abstract

In the last few years, neural network wave functions have proven effective in describing many-body quantum systems. In particular, architectures such as FermiNet, PauliNet, and PsiFormer have been successful in tackling systems of interacting electrons in quantum chemistry. However, these all rely on representing antisymmetric fermion wave functions as determinants, whose evaluation may be prohibitively expensive for large systems. A recently proposed alternative approach implements fermionic antisymmetry using a permutation symmetric neural network extended with a predefined antisymmetric function inspired by fractional quantum Hall states. In principle, such a neural network may be computationally more efficient than the standard approach.

Acknowledgements

XXX

Contents

List of Figures

List of Tables

1 Introduction

In theoretical condensed matter physics, in order to describe the properties of a material, it is usually sufficient to describe the ground state of the many-body system. However, this has always been a hard task to solve, mainly accomplished by various variational methods. It becomes particularly hard to solve quantum many-body systems which include non-trivial, strongly correlated interactions. Recently, relatively new approaches to model the wavefunction have emerged, such as Neural Quantum States (NQS)[?]. Using it as the ansatz within Variational Monte Carlo (VMC) techniques, together with a finite basis set, has significantly improved the accuracy of the method.

While the computational cost of a bosonic ansatz can scale favorably, encoding the required spin statistics for fermions introduces a major structural challenge. For bosons, one can achieve permutation invariance through simple symmetric pooling, but fermions require a totally antisymmetric wavefunction to satisfy the Pauli Exclusion Principle. This necessitates the use of the Slater determinant, which, using LU decomposition, can be calculated in $O(n^3)$, and can become a bottleneck when training systems with a large number of fermions.

Different methods have been proposed over the years to approach this, such as calculating the Slater determinant or the Pfaffian, or backflow transformations, initially proposed by R. Feynman [?], and later realised as neural backflow [?].

The Universal Approximation Theorem tells us that with a sufficiently large linear layer and a non-linear activation function we can approximate a function of arbitrary complexity, which sets the premise of this thesis. While Variational Monte Carlo pipeline with NQS training have been implemented before. Certain antisymmetric functions have been only theorized before, and this thesis sets to implement one of those antisymmetric representations and rigorously test its efficiency and general usefulness in practice.

2 Theoretical Background

2.1 Quantum Many-Body Problem in First Quantisation

Fermions are identical particles whose total wavefunction is antisymmetric under the exchange of any two particles.

- place explicit formulas for fermions $1/2$ (psi- - psi+)
- place slater determinant formula.

[?] For a discrete system of, e.g., N spins- $1/2$, the dimension of the Hilbert space is 2^N , which is exponentially large in the number of particles. Therefore, representing a quantum state of the many-body system just by giving the coefficients of the wave function in some local basis is an inefficient representation. The Hilbert space of a quantum many-body system is a really big place with an incredibly large number of quantum states. Not all quantum states in the Hilbert space of a many-body system are equal: some are more relevant than others. In particular, one can prove that low-energy eigenstates of gapped Hamiltonians with local interactions obey the so-called area-law for the entanglement entropy. This means that the entanglement entropy of a region of space tends to scale, for large enough regions, as the size of the boundary of the region and not as the volume. And this is a very remarkable property, because a quantum state picked at random from a many-body Hilbert space will most likely have an entanglement entropy between subregions that will scale like the volume, and not like the area. In other words, low-energy states of realistic Hamiltonians are not just “any” state in the Hilbert space: they are heavily constrained by locality so that they must obey the entanglement area-law.

(- paste figure of target hilbert space)

[?]

Litterture on the topic has well documented Fermionic Sign Problem, which appears in VMC setups as a consequence of the Pauli principle and constitutes a big computational challenge for VMC setups. However in our case it is an irrelevant concern (BECAUSE, short explanation). Hence we completely focus on the cost scaling for the antisymmetric part.

formal mathematics of the permutation symmetric network extended with the predefined antisymmetric function.

However in our case we work with continuous space, meaning that arguments to our Wave-function are directly coordinates of the particle. - other implications of continuous space

2.2 Variational Monte Carlo

Since we work in continuous, high dimensional, infinite space we can not perform integration to evaluate our wavefunctions. Main approach and framework which allows us to postulate wavefunction Ψ , estimate its local energies, their variance etc by statistical and computational methods is Variational Monte Carlo (VMC).

The base of VMC is the quantum-mechanical variational principle, one of the big advantages of which is that the energy of the system is bounded from below by the ground state energy. By optimizing ?? our variational state with respect to parameters θ in order to decrease the energy, we hope to capture an approximation of the ground state, hence capture the correct ground-state behavior

The known limitation of the method is its bias to Ansatz, the first assumption of the wavefunction we start optimisation with. Hence over last 10 years Neural Network Ansatz ?? became increasingly more popular as it is almost always more efficient then starting from the random state.

Assuming states are orthogonal and normalised, (claude rewrite those equations Then, any quantum state $|\psi\rangle$ can be written as: $|\psi\rangle = \int \psi(\mathbf{x}) |\mathbf{x}\rangle d\mathbf{x}$. (5.2) In turn, the expectation value of an operator O over a given variational wave function $|\psi\rangle$ takes the following form: $\langle O \rangle = \langle \psi | O | \psi \rangle = \int \int \psi^*(\mathbf{x}) \langle \mathbf{x} | O | \mathbf{x}' \rangle \psi(\mathbf{x}') d\mathbf{x} d\mathbf{x}'$. (5.3) $\langle O \rangle = \int \psi^*(\mathbf{x}) O(\mathbf{x}) \psi(\mathbf{x}) d\mathbf{x}$, (5.4) where we have defined the local estimator of the operator O : $OL(\mathbf{x}) = \langle \mathbf{x} | O | \mathbf{x} \rangle$. (5.5) The important point is that $P(\mathbf{x}) = |\psi(\mathbf{x})|^2$

Therefore, the problem of computing a quantum average of the operator O can be rephrased into the calculation of the average of the random variable $OL(\mathbf{x})$ of Eq. (5.5) over the distribution probability $P(\mathbf{x})$ given by Eq. (5.6).

summary $\langle O \rangle = \frac{1}{N} \sum_{n=1}^N OL(\mathbf{x}_n)$.

2.3 Neural Network Ansatz for Fermions

- introduce general idea that VMC WF is NN !

For fermions, efficiently representing antisymmetric states remains a major challenge. The standard approach is to expand Ψ in terms of Slater determinants constructed from a single-particle basis set $\phi_1(r), \dots, \phi_L(r)$. The problem with the Slater determinant is that it scales exponentially with the number of particles N . From the perspective of computational cost, it translates to $O(n^3)$ complexity, when solving the Slater determinant with LU decomposition. The main goal of this thesis is to verify an ansatz alternative to the Slater determinant, which could bring down the computational cost to $O(n^2)$, thus being a more efficient representation for fermionic wavefunctions. Such an ansatz was first proposed to use a symmetrical part and an antisymmetric part as arguments to a neural network [?].

$$\psi(r_1, \dots, r_n) = \Psi(r_1, \dots, r_n, \eta(r_1, \dots, r_n)) \quad (1)$$

Where Ψ is implemented as a Neural Network. While we know how to make a symmetrized Ansatz for the bosonic case (Section ??), the challenge is to construct the so-called *signature encoder*:

$$\eta = \prod_{1 \leq j < i \leq N} (x_i - x_j) \quad (2)$$

This can be extended to two dimensions using complex coordinates.

$$\eta = \eta_1 + i\eta_2 = \prod_{1 \leq j < i \leq N} (z_i - z_j) \quad (3)$$

Later in this work we use a regularised version of η for stability at the nodes, i.e. when $|z_i - z_j| \rightarrow 0$:

$$\eta = \prod_{1 \leq j < i \leq N} \frac{z_i - z_j}{\sqrt{|z_i - z_j|^2 + a^2}} \quad (4)$$

2.3.1 Stochastic Reconfiguration Driver (SR)

A big part of the VMC setup is the VMC driver. We use NetKet's implementation of the `VMC_SR` driver and this section will explain it.

Generally, the driver defines a training loop which generates samples with Monte Carlo, evaluates the local energies E_{loc} , calculates the gradients, and updates the parameters θ .

The driver we use is called the Stochastic Reconfiguration Driver, with the SR preconditioner [?, ?]. The role of SR is to precondition the gradient used in stochastic optimization, improving the convergence rate to the ground state of a Hamiltonian by multiplying the gradients with the inverse of the S matrix.

The S matrix is actually the Quantum Geometric Tensor (QGT). While standard ML optimizers like Adam work in Euclidean parameter space and use the Euclidean norm, the QGT is equipped with a quantum-mechanical distance function, the Fubini-Study distance. To construct the SR update we will first need the gradient with respect to the variational parameters θ .

We are concerned with the problem of calculating the optimal update to θ , and thus need to solve

$$\delta\theta = S^{-1}g \quad (5)$$

Since the S matrix is symmetric and positive semi-definite, we use Cholesky decomposition to solve for the inverse of the system. Here we need to introduce a parameter we will use later, called the diagonal shift. It helps with ill-conditioned S matrices.

Explain its parameters

The bottleneck of fitting the model on a GPU is the S matrix of the Stochastic Reconfiguration driver, which has dimensions $P \times P$, where P is the number of parameters. The total number of parameters of the model can be calculated as:

$$P = \dots \quad (6)$$

Show connection to Jacobian matrix and how to calculate total weight

2.3.2 Deep Sets

2.3.3 Holomorphic Trap

We will discuss it more in results section ??, but following must be mentioned now. The construction above has a shortcut built into it. For any smooth symmetric function $g(\xi)$, the state

$$\psi(r_1, \dots, r_N) = \eta(r_1, \dots, r_N) \cdot g(\xi(r_1, \dots, r_N)) \cdot e^{-\sum_i r_i^2/2} \quad (7)$$

is automatically antisymmetric, since η already flips sign under every particle exchange. Thus network does not have to learn anything about the sign structure to satisfy the Pauli principle – it can simply ignore the phase of η and output a plain symmetric prefactor. This creates a bias towards the holomorphic (I honestly don't like name holomorphic here) trap which, it is a property of the ansatz itself and will cause troubles in actual implementation further.

The simplest such choice, $g \equiv \text{const}$, turns out to be an exact eigenstate of the non-interacting Hamiltonian. Writing particle positions as complex numbers $z_i = x_i + iy_i$, the signature encoder for $\text{dim} = 2$ is the complex Vandermonde determinant

$$\eta = \prod_{i < j} (z_i - z_j) = \det \begin{pmatrix} 1 & 1 & \dots & 1 \\ z_1 & z_2 & \dots & z_N \\ \vdots & \vdots & & \vdots \\ z_1^{N-1} & z_2^{N-1} & \dots & z_N^{N-1} \end{pmatrix}, \quad (8)$$

which is exactly the Slater determinant built from the single-particle orbitals $1, z, z^2, \dots, z^{N-1}$. Each orbital z^k is an eigenstate of the 2D harmonic oscillator with energy $\hbar\omega(k+1)$, since it is purely holomorphic and carries all k of its quanta as angular momentum ($n = n_x + n_y = k$). Filling the lowest N of them (aufbau) gives the total energy of this state as

$$E_{\text{lazy}} = \hbar\omega \sum_{k=0}^{N-1} (k+1) = \hbar\omega \frac{N(N+1)}{2}. \quad (9)$$

Because E_{lazy} is a true eigenvalue of $\hat{H}$, this state is an exact stationary point of the energy expectation value $\langle \psi | \hat{H} | \psi \rangle / \langle \psi | \psi \rangle$ [?]: the energy gradient vanishes there identically, for any architecture, no matter what it is in principle capable of representing elsewhere. Gradient-based VMC has no local incentive to leave this state once it lands there. And since $g \equiv \text{const}$ is the easiest possible function to reach from a random initialization, it lands there almost every time. We call this the *holomorphic trap*: not a failure of expressivity, but of trainability – the network can represent the true ground state (we show this directly in Section ??), it just has no gradient telling it to go there.

Nothing in this argument is specific to the isotropic harmonic trap. The trapped family, more generally, consists of any state whose antisymmetric part is a product of pairwise coordinate differences – what we call a *sortable* sign structure, since it is exactly the sign structure a sorting of the particles would produce. The true 2D ground state instead has a sign structure built from a signed area (e.g. $\det\{1, x, y\}$ for $N = 3$), which cannot be factored into pairwise terms. It is this genuinely two-dimensional antisymmetry, not the harmonic potential specifically, that the ansatz fails to reach unaided – we confirm this by repeating the experiment on a potential with no rotational symmetry to lean on, in Section ???. That section also presents two structural ways to escape the trap without ever knowing the true ground state in advance.

3 Implementation and Experimental Setup

System (Hamiltonian) $\rightarrow$ Sampler (Metropolis Algo) $\rightarrow$ Ansatz (NQS) $\rightarrow$ VMC (Eigen energies)

3.1 System Architecture

3.2 The Sampler

// explain what metrics we use for looking at the problem, Rhat, TauCorr etc

3.3 System

// mb remove completely

The object **System** includes a description of the continuous Hilbert space we work with, its dimension, and the Hamiltonian (namely the potential part) for which we solve Schrödinger's equation. The potential of our choice, to begin with, is the Quantum Harmonic Oscillator. It's worth noting that we enforce antisymmetrization of the state into our Ansatz (Neural Quantum State).

The configuration file for this experiment (**Experiment**¹) and the ansatz implementation (**Ansatz**²) are both available online.

Simulations for this work were performed using NetKet [?, ?]. This software is built on top of JAX [?] and Flax [?].

3.4 Neural Network Ansatz

As was introduced in Section ?? ...

The size of our input layer is defined by several parameters.

- to avoid correlation between the Markov chains we increase **n_chains**, which determines the number of chains we spawn

- **n_samples** defines the number of samples generated by each chain

- **N** is the number of particles in the system

- **dim** is respectively the dimension of the continuous Hilbert space

- an additional computational parameter is **chunk_size** which allows us to fit the model on the GPU.

input_dimension = **n_chains** $\times$...

Other variables in the ansatz are: - **a** is a regularisation parameter for the antisymmetric function.

Another hurdle of the computational implementation is that we always have to treat the output of our Ansatz as a logarithm.

¹https://github.com/IlyaYakushevskiy/FerminQS/blob/main/configs/experiment/qho_fermisets_2d_5N_post.yaml

²<https://github.com/IlyaYakushevskiy/FerminQS/blob/main/src/ansatz.py>

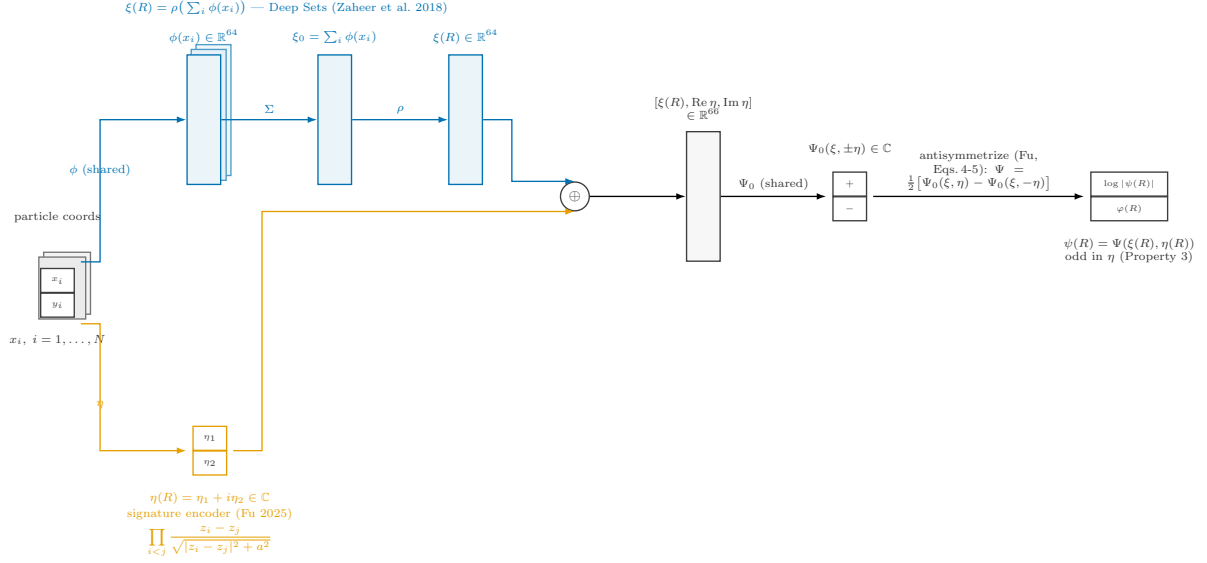


Figure 1: FermiSets architecture

4 Results & Discussion

For the system discussed in Section ?? we are going to do an ablation study, according to [?].

We are going to start with the simplest system, 4 spinless fermions in a Quantum Harmonic Oscillator potential, with the following Hamiltonian, to test our ansatz:

$$\hat{H}_0 = \sum_{i=1}^4 \left[-\frac{\hbar^2}{2m} \nabla_i^2 + \frac{1}{2} m \omega^2 \mathbf{r}_i^2 \right] \quad (10)$$

Let us look first at the 2D case: we define $m\omega^2 = 1$, so we can find the analytic ground state energy as $\hbar\omega(n+1)$. The problem with training the network for 4 fermions arises from the threefold degenerate ground state.

From an analysis of the possible single-particle wavefunctions with $n = n_x + n_y$, we can see the closest states to the ground state will be:

- (n = 0) : (0,0)
- (n = 1) : (0,1), (1,0)
- (n = 2) : (0,2), (2,0), (1,1)
- (n = 3) : (1,2), (2,1), (3,0), (0,3)

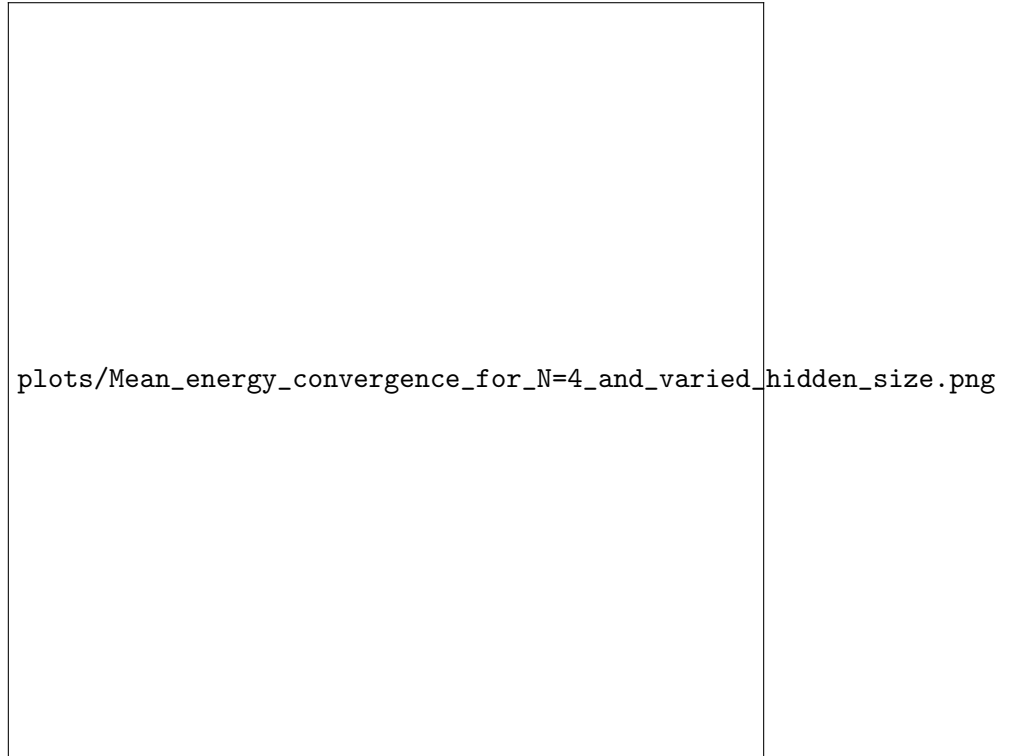
So the true ground state would be of the form:

$$\Psi_{\text{GS}} = c_1 \Psi_{2,0} + c_2 \Psi_{1,1} + c_3 \Psi_{0,2} \quad (11)$$

and the Energy of this system would be :

$$E = \hbar\omega(n_x + n_y + \frac{1}{2} + \frac{1}{2}) = \hbar\omega((0+0+1) + (0+1+1) + (1+0+1) + (0+2+1)) = \hbar\omega 8 \quad (12)$$

From our energy convergence graph, Figure ??, we see that all configurations are heavily biased to the state with $E = 10$, which, as testing with different N confirms, is always the exact Laughlin state.

Figure 2: *My caption*

4.1 Test of Optimizers

Since we had significant troubles with instabilities within the SR driver, we also tested the very standard, general Machine Learning approach of using the Adam Optimizer without Stochastic Reconfiguration. Since we don't have to compute the S matrix, that allowed us to fit a much bigger model on the same GPU. Instead of having [16, 32, 64] hidden units we used [16, 32, 1024], and additionally added one extra layer with weight dimension $[\text{hidden_size} \times \text{hidden_size}]$.

We report that adding the extra layer (even when reduced in size) increased training time massively (30 times more per iteration), which we attribute to the way the code is parallelized in our setup.

The setup with Adam and no SR did not have numerical instability troubles and found the Laughlin state with low errors of around 0.001 percent, but couldn't converge to the actual ground state.

Solver for SR Optimizer However, the inverse is often ill-defined. One reason is that Monte Carlo estimates of matrix elements are noisy. Noise accumulates to render the matrix singular by making small eigenvalues vanish. Therefore, quickly and efficiently obtaining many uncorrelated samples is crucial. [?]

4.2 Hyperparameter Optimization

Effect of hidden layer size The best `diag_shift` is 0.001, with the soft-SVD optimiser.

Regularisation of antisymmetric part In our formulation of the antisymmetric function, Equation ??, which is regularised by the parameter a , we made a quick sweep to assess its effect:

4.3 Performance of the best model

The best result is from run 2026-05-14/18-38-51

graph for variance – refer to zero variance principle Overlap over time - oscillating behaviour
? TODO plot against Laughlin state

main problem is characterised by smooth descent -> jump -> smooth descent

an idea for it: "Continuous functions cannot just teleport their zero-crossings. To fix the geometry, the network has to literally tear and reconnect these multidimensional nodal surfaces.

During the steps where the nodes are actively shifting and crossing through regions of high particle density, the kinetic energy violently spikes. Once the nodes snap into the correct new topology, the energy collapses back down and resumes a smooth descent toward a lower plateau."

-> read on mitigation strategies - energy clipping - metropolis exchange

print MCState PARAMETERS

LIST THE PLOTS

EXPORT OUTPUTS / folder ON HDD

INCLUDE 1D EXPL / COMPTA TO 2D

4.4 Laughlin State (MAIN MEAT3)

Classically the Lowest Landau Level Laughlin state is $\Psi_{LL} = \text{Vandermonde} \cdot \text{Boson} \cdot \text{Gaussian}$ and it is a state constructed to represent an electron gas placed in a uniform background magnetic field in the presence of a uniform jellium background.

Empirically it also appears to be a "lazy" state which our ansatz (Section ??) converges to.
// go back to L. Fu and write down the ansatz

Proof:

Theory: It gets stuck in the Laughlin state, compute overlap with the Laughlin state, compute overlap between (state, state + 50 iters)

4.5 Supervised pretraining

As we saw in the previous section, the shape of our Ansatz leads VMC to get stuck in the Laughlin eigenstate trap. The natural question was whether the expressivity of the Ansatz was the problem. We thus performed supervised pretraining for our 2D QHO setup, and after training the eigenenergy converged to the energy of the real ground state. This rules out the idea that the Ansatz lacks the expressivity to replicate the structure of the Slater determinant.
ipaste the plot

4.6 Results with radial symmetry assumption

Where we finally achieved our first success was in introducing radial symmetry for the 2D QHO; this is a special case which works when the ground state is radially symmetric. This means we must have a spherically symmetric potential, and the GS must be non-degenerate.

We can thus work with systems of $N = 3, 6, 10, \dots$ particles. Later we will test it on a quantum dot system, where we encode the Coulomb interaction (Section ??).

For such a system we know for a fact that $[\hat{H}, \hat{L}_z] = 0$, meaning that eigenstates of the Hamiltonian can also be chosen as eigenstates of the $\hat{L}_z$ operator.

We thus need to project our neural network onto definite sectors of $\hat{L}_z$. The way we achieve this is to apply a rotation K times, with angles $\theta_k = \frac{2\pi k}{K}$ for $k = 0, 1, \dots, K-1$, to the coordinates fed into the symmetric part (Neural Network), and then average the result. We must note that this increases the computational cost by a factor of K . Hence the cost of the ansatz is $O(K \cdot n^2)$

compared to $O(n^3)$ in the case of the Slater determinant ansatz. The scaling of the error with K and N is yet to be studied.

- $N = 3$: converges nicely.
- $N = 6$: $K = 3, 4, 6$ are slower to converge than Slater, getting stuck and slowing around $E = 17$.
- $N = 10$: blows up.

5 Conclusions & Outlook

Conclusions

Outlook

Part I

Appendix